Study Notes: Days 43–49 — React + TypeScript

These notes follow Month 2, Week 3 of the roadmap, which is **Week 7 overall**. JavaScript and TypeScript snippets are independent unless identified as application fragments. React examples assume a React + TypeScript project. The integration example uses illustrative API contracts; it does not claim that Sai implemented those exact features.

Day 43 — JavaScript fundamentals

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Reason accurately about values, equality, objects, functions, and common array operations before using them in a frontend application.

Values, variables, and equality

JavaScript has primitive values such as strings, numbers, booleans, `null`, `undefined`, `bigint`, and symbols, as well as objects. Variables hold values; when an object is assigned or passed, the reference value is copied, so multiple variables can refer to the same mutable object. `const` prevents reassignment of the variable, not mutation of the referenced object.

`number` uses IEEE 754 floating point. Decimal fractions such as `0.1` are not always represented exactly, and integers above `Number.MAX_SAFE_INTEGER` cannot all be represented precisely. For financial calculations, choose an explicit representation and rounding policy, such as integer minor units within a safe range or a suitable decimal library and API contract. `bigint` supports large integers but cannot be mixed directly with `number` arithmetic and needs deliberate JSON handling.

Use `===` and `!==` for equality without coercion in ordinary code. Object equality is reference identity, not deep comparison. `NaN !== NaN`; `Number.isNaN` detects it without coercing other values. `Object.is` differs from `===` for `NaN` and signed zero. `null` and `undefined` both represent absence in different contexts; `??` supplies a fallback only for those values, whereas `||` also replaces `0`, `false`, and `\"\"`.

Functions, objects, and `this`

Functions are values: they can be passed as callbacks, returned from other functions, and stored in objects. A regular function's `this` depends on how it is called; an arrow function captures `this` lexically from its surrounding context. Extracting a method and calling it separately can therefore lose its receiver. Avoid relying on implicit binding when a clear function argument or explicit binding would be easier to maintain.

Objects have properties and a prototype chain used for property lookup. Array and object spread create **shallow** copies: nested objects remain shared unless separately copied. Mutation is not inherently forbidden in JavaScript, but shared mutation makes reasoning and change detection harder, particularly with React state.

Arrays and transformations

`map` creates a new array from a projection; `filter` selects items; `find` returns the first matching item or `undefined`; and `reduce` accumulates a result. Methods such as `push`, `splice`, and `sort` mutate the array. To

sort without mutating an existing array, copy it first or use `toSorted` where supported. The default `sort` compares string representations, so numbers usually require a comparator.

Choose the operation that communicates intent and consider allocation and complexity in hot paths. Avoid using `map` solely for side effects, and provide an explicit initial value to `reduce` when empty input must be handled.

Code example

```
const original = { name: "Ada", address: { city: "Pune" } };
const copy = { ...original };
copy.address.city = "Hyderabad";
console.log(original.address.city); // Hyderabad: nested object is shared

console.log(0 || 10); // 10
console.log(0 ?? 10); // 0
console.log(Number.isNaN(NaN)); // true

const numbers = [10, 2, 1];
const sorted = [...numbers].sort((a, b) => a - b);
console.log(sorted);  // [1, 2, 10]
console.log(numbers); // [10, 2, 1]: original was not sorted in place

const total = numbers.reduce((sum, value) => sum + value, 0);
console.log(total); // 13
```

DSA problem: Reverse a linked list

Problem: Given the head of an acyclic singly linked list, reverse its links in place and return the new head.

Approach: Keep `previous` and `current` pointers. Save the next node before redirecting the current node's link, then advance both pointers.

Complexity: O(n) time and O(1) additional space. The input list is mutated.

```
class ListNode {
  constructor(public value: number, public next: ListNode | null = null) {}
}

function reverseList(head: ListNode | null): ListNode | null {
  let previous: ListNode | null = null;
  let current = head;
  while (current !== null) {
    const next: ListNode | null = current.next;
    current.next = previous;
    previous = current;
    current = next;
  }
  return previous;
}
```

```
const head = new ListNode(1, new ListNode(2, new ListNode(3)));
const reversed = reverseList(head);
console.log(reversed?.value, reversed?.next?.value); // 3 2
```

Edge cases: An empty list returns `null`; a one-node list is unchanged. Forgetting to save `next` loses the rest of the list. This solution assumes no cycle; an unhandled cycle would prevent termination.

Likely interview questions

1. What is copied when an object is assigned to another variable?
2. What does `const` guarantee, and what can still be mutated?
3. How do `===`, `Object.is`, and `Number.isNaN` differ?
4. When does `??` behave differently from `||`?
5. Why can ordinary JavaScript numbers be problematic for financial values?
6. How does an arrow function's `this` differ from a regular method's?
7. Which common array methods mutate their input?
8. Why must a linked-list reversal save the next pointer before changing a link?

Common pitfalls / misconceptions

- Assuming object spread deep-copies nested objects.
- Using `||` when zero or an empty string is a valid value.
- Sorting numbers with the default string comparison.
- Losing the remainder of a linked list by overwriting `next` too early.

Cheat-sheet recap

- Objects are shared through copied references; spread is shallow.
- Prefer explicit equality and nullish handling; understand floating-point limits.
- Select array operations for their semantics and mutation behavior.
- Reverse a list with previous, current, and saved-next pointers.

Day 44 — Closures + scope + hoisting

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Explain how lexical environments and declaration rules affect callback behavior, lifetime, and correctness.

Scope

Scope determines where a binding can be referenced. JavaScript uses lexical scope: a function's access to outer bindings is determined by where it is defined, not by where it is later called. `let` and `const` are block-scoped; `var` is function-scoped or global-scoped depending on where it is declared. Modules have their own scope.

Shadowing occurs when an inner binding has the same name as an outer one. It can be useful but can also obscure which value is being read. Prefer clear names and narrow scopes so that callback and state behavior

remains understandable.

Closures

A closure is a function together with access to bindings from its lexical environment. The function can continue to access those bindings after the outer function returns. Closures enable factories, callbacks, private state, and event handlers. A closure generally observes the captured **binding**, not a snapshot of its initial value.

Capturing mutable state can cause surprising asynchronous behavior. `let` in a `for` loop creates a per-iteration binding, while a `var` loop normally shares one function-scoped binding. In React, each render creates closures over that render's values; a callback can therefore observe an older render's state if dependencies or update strategy are wrong. Closures can also keep objects reachable longer than intended; remove listeners and subscriptions when their lifetime ends.

Hoisting and the temporal dead zone

Declarations are processed before ordinary execution in their scope, but not all bindings are usable in the same way. A function declaration is generally available before its textual declaration within its scope. A `var` binding is initialized to `undefined` before its assignment executes. `let` and `const` bindings exist but cannot be accessed before initialization: that interval is the **temporal dead zone**, and access throws `ReferenceError`.

Do not treat “hoisting” as literal movement of source lines or assume every declaration behaves like `var`. Function expressions assigned to `let` or `const` follow the variable's initialization rules. Block-level function declaration behavior can differ in non-strict legacy code; prefer predictable declarations.

Code example

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}

const counter = makeCounter();
console.log(counter(), counter()); // 1 2

const callbacks = [];
for (let i = 0; i < 3; i++) {
  callbacks.push(() => i);
}
console.log(callbacks.map(callback => callback())); // [0, 1, 2]

console.log(declaredLater()); // available before its declaration
function declaredLater() { return "ready"; }

console.log(value); // undefined: var binding exists, assignment has not run
var value = 5;
```

Accessing a `let` or `const` binding before its initialization would throw; it is omitted from the runnable sequence so the example can complete.

Likely interview questions

1. What is lexical scope, and how is it different from the call stack?
2. What does a closure retain after its outer function returns?
3. Does a closure capture a frozen value or a binding?
4. Why do `let` and `var` behave differently in a loop that creates callbacks?
5. What is the temporal dead zone?
6. How do function declarations and function expressions differ before initialization?
7. How can closures retain memory longer than intended?
8. How can a React callback observe state from an earlier render?

Common pitfalls / misconceptions

- Saying a closure copies all outer values at creation.
- Assuming `var`, `let`, `const`, and function declarations have identical hoisting behavior.
- Keeping long-lived subscriptions or callbacks without considering captured objects.
- Treating a stale React closure as a random timing problem rather than a dependency or state-update issue.

Cheat-sheet recap

- Scope is lexical; closures retain access to outer bindings.
- `let`/`const` are block-scoped and have a temporal dead zone.
- `var` is function-scoped and initialized to `undefined` before assignment.
- Manage closure lifetime and use appropriate dependencies or functional state updates.

Day 45 — Promises + event loop

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Predict asynchronous ordering, propagate promise failures, and distinguish concurrency from the JavaScript execution thread.

Promises and async/await

A promise represents the eventual outcome of an operation. It can be pending, fulfilled, or rejected; once settled, its outcome does not change. `then` returns a new promise and can transform or adopt the result of another promise. `catch` handles rejection in the chain, and `finally` runs cleanup without receiving the result; a failure in cleanup can replace the prior outcome.

An `async` function returns a promise. `await` pauses that function until the awaited value settles and then resumes through promise continuation machinery; a rejection is thrown at the await point. Handle errors where the caller can make a meaningful decision. `fetch` rejects for certain network failures, but HTTP 404 or 500 normally resolves to a `Response`, so check `response.ok`.

`Promise.all` waits for all inputs to fulfill and returns results in input order; it rejects when an input rejects but does not automatically cancel other work. `Promise.allSettled` reports each outcome. `Promise.race` and

`Promise.any` have different completion semantics. Use an `AbortController` or another explicit mechanism when supported; canceling a request does not prove that the server did not already process it.

The event loop

JavaScript executes synchronous code on a call stack within an agent. The host provides facilities such as timers, network I/O, and task scheduling. Promise reactions are queued as **microtasks**; in a browser, after a task's synchronous work completes, the host processes the microtask queue before moving on according to the event-loop rules. Excessive microtasks can delay rendering and other work.

Timers schedule work no earlier than the requested delay; `setTimeout(..., 0)` is not an instruction to run immediately. Browser rendering and task-source scheduling have their own rules, and Node.js has host-specific phases and additional scheduling mechanisms. Do not generalize one small ordering example into a universal order for every I/O callback.

Code example

```
console.log("start");
setTimeout(() => console.log("timer"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("end");
// For this simple example: start, end, promise, timer

async function loadExample() {
  const response = await fetch("/api/example");
  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }
  return response.json();
}

// A caller should await loadExample() and handle or propagate its rejection.
```

The first part demonstrates basic browser/Node ordering; the second requires a host with `fetch` and a reachable endpoint. It defines a function without making a network request.

DSA problem: Find the middle node

Problem: Return the middle node of an acyclic singly linked list. For an even-length list, return the **second** middle node. Return `null` for an empty list.

Approach: Move a slow pointer one node and a fast pointer two nodes at a time. When the fast pointer reaches the end, the slow pointer is at the chosen middle.

Complexity: O(n) time and O(1) additional space.

```
class ListNode {
  constructor(public value: number, public next: ListNode | null = null) {}
}
```

```ts
function middleNode(head: ListNode | null): ListNode | null {
  let slow = head;
  let fast = head;
  while (fast !== null && fast.next !== null && slow !== null) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow;
}

const head = new ListNode(1, new ListNode(2, new ListNode(3, new ListNode(4))));
console.log(middleNode(head)?.value); // 3: second middle
```

Edge cases: Empty input returns `null`; a single node is its own middle; an even list returns the second middle by this contract. The list must be acyclic.

Likely interview questions

1. What is the difference between a pending, fulfilled, and rejected promise?
2. How do `async`/`await` relate to promise continuations?
3. Why must code check `response.ok` after `fetch`?
4. What happens to other operations when `Promise.all` rejects?
5. Why does a promise callback run before a zero-delay timer in the shown example?
6. Can `setTimeout(..., 0)` guarantee immediate execution?
7. What can excessive microtasks do to responsiveness?
8. Why does the slow/fast pointer algorithm choose the second middle for an even list?

Common pitfalls / misconceptions

- Assuming `fetch` rejects for every non-2xx response.
- Starting promises without observing their failures.
- Assuming `Promise.all` cancels sibling operations or that aborting proves server rollback.
- Treating a simple event-loop ordering example as a guarantee for every host and I/O case.

Cheat-sheet recap

- `async` functions return promises; `await` propagates rejection at the await point.
- Promise reactions are microtasks; timers are scheduled host tasks, not immediate calls.
- Coordinate concurrency, failures, and cancellation explicitly.
- Slow/fast pointers find the specified middle in one pass.

Day 46 — TypeScript: interface, type, union, generics, type guards

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Model contracts precisely, narrow values safely, and understand what TypeScript does and does not guarantee at runtime.

`interface` and `type`

TypeScript's type system is primarily structural: a value is compatible when it has the required shape, subject to language rules. An `interface` describes an object-like contract and supports extension and declaration merging. A `type` alias can name object shapes, unions, intersections, tuples, primitives, and other types. Either can describe many object contracts; choose based on the shape and project conventions rather than claiming one is always superior.

Excess-property checks on fresh object literals are useful but do not make TypeScript nominal or validate arbitrary runtime objects. Private or protected class members and other special cases affect compatibility. Type information is erased from emitted JavaScript, so interfaces and aliases do not validate API responses.

Unions and narrowing

A union such as `Loading | Success | Failure` models a value that can be one of several alternatives. A discriminant like `status` lets control flow narrow the available fields. This is often clearer than an object with many optional properties that permit impossible combinations.

Narrowing uses checks such as `typeof`, `in`, equality, `instanceof`, or a custom guard. `instanceof` depends on runtime constructors and can be unreliable across realms or for plain JSON. Handle every union variant; an exhaustive `never` check can reveal missing cases when the union grows.

Generics

Generics express relationships between input and output types, such as a function returning the same element type it receives. Constraints such as `T extends { id: string }` state what the generic implementation may safely use. A generic does not automatically validate external data or guarantee that a runtime value matches a caller-supplied type argument.

Use generics when the relationship is meaningful. Overly complicated generic types can make an API harder to understand; sometimes a concrete type or union is clearer.

Type guards and runtime boundaries

A type guard performs a runtime check and tells the compiler how to narrow the value. A user-defined predicate is only as trustworthy as its implementation: TypeScript does not prove that the function checks every required property. At a network or storage boundary, start with `unknown`, validate required fields and domain constraints, and only then use the value as the intended type. For large schemas, a runtime validation library can make the contract more maintainable.

Avoid `as SomeType` merely to silence an error: assertions change the compiler's view, not the actual data. Type checking and runtime validation address different risks.

Code example

```
interface Deposit {
  id: string;
  amount: number;
}

type LoadState<T> =
  | { status: "loading" }
```

```
  | { status: "success"; data: T }
  | { status: "error"; message: string };

function hasId<T extends { id: string }>(value: T): string {
  return value.id;
}

function isDeposit(value: unknown): value is Deposit {
  return typeof value === "object" && value !== null
    && "id" in value && typeof value.id === "string"
    && "amount" in value && typeof value.amount === "number"
    && Number.isFinite(value.amount);
}

const incoming: unknown = JSON.parse('{"id":"d1","amount":100}');
const state: LoadState<Deposit> = isDeposit(incoming)
  ? { status: "success", data: incoming }
  : { status: "error", message: "Invalid deposit response." };

if (state.status === "success") {
  console.log(hasId(state.data), state.data.amount); // d1 100
} else if (state.status === "error") {
  console.error(state.message);
}
```

The guard validates this example's declared shape, not every financial rule; an actual API may also require currency, precision, range, and authorization checks.

Likely interview questions

1. How do interfaces and type aliases overlap, and where do they differ?
2. What does structural typing mean?
3. Why can a discriminated union be safer than several optional fields?
4. How do generic constraints help an implementation?
5. What is the difference between `unknown` and an unchecked type assertion?
6. Why does a generic API response type not validate the JSON at runtime?
7. What makes a user-defined type guard trustworthy?
8. How would you make a union switch exhaustive?

Common pitfalls / misconceptions

- Assuming interfaces exist as runtime validators.
- Using `as` to force untrusted JSON into a type without checking it.
- Writing a guard that checks only some required properties.
- Creating unnecessary generic complexity instead of modeling a clear contract.

Cheat-sheet recap

- Interfaces and type aliases describe compile-time shapes; TypeScript is largely structural.
- Discriminated unions represent mutually exclusive states.

- Generics express relationships; constraints describe required capabilities.
- Validate `unknown` at runtime boundaries before trusting it.

Day 47 — React rendering + state

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Explain render and commit, state snapshots, immutable updates, and how component identity affects state.

Render and commit

React renders a component by calling it to compute a description of the UI from props, state, and context. React compares the result with the previous tree and commits necessary changes to the host environment, such as the DOM. A render does not necessarily mean every DOM node changes. Rendering should be pure: React may call components more than once, interrupt or discard work in some circumstances, and perform extra development checks under Strict Mode.

State belongs to a component's position and identity in the rendered tree, not to a local variable inside one function call. Changing the component type or its key can reset state. Keys help React match siblings across updates; use stable identifiers from the data, not array indices when items can be reordered, inserted, or deleted.

State snapshots and updates

Each render sees a snapshot of state. Calling a setter requests another render; it does not mutate the state variable captured by the current handler. React batches many updates. When the next value depends on the previous value, use a functional updater such as `setCount(c => c + 1)` so queued updates compose correctly.

Treat objects and arrays in state as immutable: create new references for the changed path. A shallow copy of the outer object does not copy nested objects, so update each modified level. Mutating existing state can make updates hard to observe and can corrupt previous render assumptions. Keep derived values computed from existing state where practical rather than maintaining duplicate state that can drift.

Ownership and tradeoffs

Place state at the lowest common owner that needs to coordinate it. Local state is often simpler than global state; shared or server state may need a different mechanism. Preserve a single source of truth for each piece of data. Controlled form inputs derive their values from React state; uncontrolled inputs keep their current value in the DOM and can be managed by a form library.

React can skip some updates when values are unchanged according to its comparison rules, but do not rely on mutation or undocumented scheduling as an optimization. Measure real rendering costs before adding memoization.

Code example

```tsx
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount(current => current + 1);
    setCount(current => current + 1);
    setCount(current => current + 1);
  }

  return (
    <button type="button" onClick={addThree}>
      Count: {count}
    </button>
  );
}
```

Each click queues three updates based on the previous queued value. Calling `setCount(count + 1)` three times in the same handler instead uses the same captured `count` for each call and ordinarily results in one increment.

DSA problem: Nth node from the end

Problem: Return the `n`th node from the end of an acyclic singly linked list, with `n` starting at 1. Return `null` if `n` exceeds the list length; reject nonpositive or non-integer `n`.

Approach: Advance a fast pointer `n` steps. Then move fast and slow pointers together until fast reaches the end; slow is `n` nodes from the end.

Complexity: O(length) time and O(1) additional space.

```ts
class ListNode {
  constructor(public value: number, public next: ListNode | null = null) {}
}

function nthFromEnd(head: ListNode | null, n: number): ListNode | null {
  if (!Number.isInteger(n) || n <= 0) {
    throw new RangeError("n must be a positive integer.");
  }

  let fast = head;
  for (let i = 0; i < n; i++) {
    if (fast === null) return null;
    fast = fast.next;
  }

  let slow = head;
  while (fast !== null && slow !== null) {
    fast = fast.next;
```

```
    slow = slow.next;
  }
  return slow;
}

const head = new ListNode(1, new ListNode(2, new ListNode(3)));
console.log(nthFromEnd(head, 2)?.value); // 2
console.log(nthFromEnd(head, 4));        // null
```

Edge cases: `n = 1` returns the last node; `n` equal to the length returns the head; empty input or `n` larger than the length returns `null`. Invalid `n` throws. The list must be acyclic.

Likely interview questions

1. What is the difference between rendering and committing in React?
2. Why should rendering be pure?
3. What does it mean for state to be a snapshot?
4. Why do functional state updaters matter for multiple queued updates?
5. How do keys affect component identity and state?
6. Why can a shallow copy still leave nested state shared?
7. When should state stay local rather than move to a shared store?
8. How does the two-pointer gap find the nth node from the end?

Common pitfalls / misconceptions

- Assuming every render causes a DOM update.
- Mutating nested state or duplicating derived state unnecessarily.
- Using unstable keys and then attributing state resets to randomness.
- Treating `n = 0` as valid without defining the linked-list indexing contract.

Cheat-sheet recap

- Render computes UI; commit applies changes; render logic must be pure.
- State is a render snapshot; use functional updaters for previous-state-dependent changes.
- Preserve stable component identity and update state immutably.
- Advance a fast pointer `n` steps, then move both pointers together.

Day 48 — Hooks

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Use hooks according to their rules, synchronize external systems safely, and avoid stale closures and unnecessary effects.

Rules and common hooks

Hooks let function components use React features. Call hooks at the top level of React components or custom hooks, in the same order on every render—not inside conditions, loops, nested functions, or ordinary utility functions. React relies on consistent call order to associate hook state with a component.

`useState` manages local state; `useReducer` can make complex state transitions explicit; `useRef` holds a mutable value that survives renders without triggering a render when changed. A ref is useful for a DOM node or an imperative handle, not for data that must update the visible UI. Custom hooks share stateful **logic**, not one shared state instance automatically.

Effects and cleanup

`useEffect` synchronizes a component with an external system after commit: examples include subscriptions, timers, and network operations. React runs cleanup before rerunning an effect whose dependencies changed and when the component unmounts. In development Strict Mode, React may run an extra setup/cleanup cycle to expose unsafe assumptions. Effects should tolerate correct cleanup and re-setup.

The dependency list must include reactive values used by the effect; React compares dependencies with `Object.is`. Omitting a dependency to silence reruns can create stale closures. If there is no external system to synchronize, an effect may be unnecessary: derive values during render or perform user-triggered work in an event handler instead.

Async work and races

An effect callback itself should not be `async` because React expects either a cleanup function or nothing, not a promise. Define an async function inside it, invoke it, and handle rejection. Abort or otherwise ignore obsolete requests during cleanup so an older response does not overwrite a newer selection. `AbortController` can stop supported client-side work but cannot guarantee the server did not process a request.

Effects are not a complete data-fetching architecture: caching, deduplication, server rendering, retries, and consistency may be better handled by a framework or query library. Still understand the lifecycle, because those libraries rely on the same underlying concerns.

Code example

```
import { useEffect, useState } from "react";

type Status =
  | { kind: "loading" }
  | { kind: "ready"; name: string }
  | { kind: "error"; message: string };

function hasName(value: unknown): value is { name: string } {
  return typeof value === "object" && value !== null
    && "name" in value && typeof value.name === "string";
}

export function UserName({ userId }: { userId: string }) {
  const [status, setStatus] = useState<Status>({ kind: "loading" });

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      setStatus({ kind: "loading" });
```

```
        try {
          const response = await fetch(`/api/users/${encodeURIComponent(userId)}`, {
            signal: controller.signal,
          });
          if (!response.ok) throw new Error(`Request failed: ${response.status}`);
          const body: unknown = await response.json();
          if (!hasName(body)) throw new Error("Invalid user response.");
          if (!controller.signal.aborted) setStatus({ kind: "ready", name: body.name
});
        } catch (error: unknown) {
          if (!controller.signal.aborted) {
            setStatus({
              kind: "error",
              message: error instanceof Error ? error.message : "Request failed.",
            });
          }
        }
      }

      void load();
      return () => controller.abort();
    }, [userId]);

    if (status.kind === "loading") return <p>Loading...</p>;
    if (status.kind === "error") return <p role="alert">{status.message}</p>;
    return <p>{status.name}</p>;
}
```

The endpoint is illustrative and must return an object with a string `name`. Cleanup and the abort check prevent this effect from applying an obsolete result after its request is canceled; a production application should also choose an appropriate caching and error-reporting strategy.

Likely interview questions

1. Why must hooks be called in a consistent order?
2. What is the difference between `useState` and `useRef`?
3. When is `useReducer` useful?
4. What is an effect for, and when is an effect unnecessary?
5. When does cleanup run, including in development Strict Mode?
6. Why should an effect callback not itself be `async`?
7. How do dependencies and stale closures interact?
8. How would you prevent an older network response from overwriting a newer selection?

Common pitfalls / misconceptions

- Calling hooks conditionally or treating a ref update as a render trigger.
- Omitting effect dependencies to hide a rerun.
- Using effects to copy values that could be derived during render.
- Assuming aborting a request reverses work already performed by the server.

Cheat-sheet recap

- Call hooks consistently at the top level; custom hooks share logic.
- Effects synchronize external systems and need correct dependencies and cleanup.
- Handle async failures and obsolete results explicitly.
- Prefer render-time derivation or event handlers when no external synchronization is needed.

Day 49 — RTK + React Hook Form + API integration — use things you've actually worked with

Month 2 — Full-Stack Depth, Week 3 — React + TypeScript (Week 7 overall).

Goal: Connect Redux Toolkit, React Hook Form, and an API in a maintainable feature while explaining loading, validation, caching, and failures.

Redux Toolkit (RTK)

Redux centralizes selected application state in a store. Actions describe events, reducers calculate state transitions, and subscribers receive updates. Redux Toolkit provides `configureStore`, `createSlice`, and other utilities that reduce setup and encourage consistent patterns. `createSlice` uses Immer, allowing mutation-like syntax inside case reducers while producing immutable updates; do not assume arbitrary mutation outside an Immer-managed reducer is safe.

Keep state serializable where practical so debugging and tooling remain useful; default RTK middleware warns about many non-serializable values. Select only the state a component needs. Local component or form state need not be moved to Redux merely because a store exists.

RTK Query, included with Redux Toolkit, models server queries and mutations, caches results by endpoint and serialized arguments, tracks request state, and can invalidate tags after a mutation. It reduces manual loading/error/cache code, but cache lifetime, invalidation, authentication, and runtime response validation still need design. A client cache is not the database source of truth.

React Hook Form

React Hook Form manages form registration, validation, errors, submission state, and values. Native inputs registered with `register` are commonly uncontrolled, which can reduce unnecessary form-wide renders. `Controller` can integrate components that require a controlled interface. Define default values, validate the user-facing contract, and show accessible field-level and submission errors.

Client validation improves feedback but is not a security boundary: the server must validate and authorize independently. Numeric inputs need explicit conversion and finite/range checks. For financial amounts, decide how to represent decimal precision across the browser and API; JavaScript `number` alone does not provide exact decimal arithmetic.

API integration and production tradeoffs

Define request and response contracts, loading/empty/error states, cancellation and retry behavior, and what happens after a successful mutation. A TypeScript generic for an API response is a compile-time expectation, not runtime validation of JSON. Validate untrusted responses where required, avoid exposing secrets, and handle 401/403 according to the application's authentication flow.

A timed-out or aborted mutation may already have succeeded on the server. Retrying a non-idempotent create without a server-supported idempotency strategy can duplicate it. Cache invalidation triggers a refetch for affected active queries, but does not make distributed writes atomic or guarantee immediate consistency in every backend.

Code example

This example assumes React, React Redux, Redux Toolkit, and React Hook Form are installed. It also assumes a backend exposing `GET /api/tickets` returning `Ticket[]` and `POST /api/tickets` accepting `{ \"title\": string }` and returning a created `Ticket` as JSON. It is an illustrative contract, not an existing API. Place the code in a React TypeScript component file such as `App.tsx`:

```
import { configureStore } from "@reduxjs/toolkit";
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
import { Provider } from "react-redux";
import { useForm } from "react-hook-form";

type Ticket = { id: string; title: string };
type TicketInput = { title: string };

const api = createApi({
  reducerPath: "ticketApi",
  baseQuery: fetchBaseQuery({ baseUrl: "/api/" }),
  tagTypes: ["Tickets"],
  endpoints: builder => ({
    getTickets: builder.query<Ticket[], void>({
      query: () => "tickets",
      providesTags: ["Tickets"],
    }),
    createTicket: builder.mutation<Ticket, TicketInput>({
      query: body => ({ url: "tickets", method: "POST", body }),
      invalidatesTags: ["Tickets"],
    }),
  }),
});

const store = configureStore({
  reducer: { [api.reducerPath]: api.reducer },
  middleware: getDefaultMiddleware =>
getDefaultMiddleware().concat(api.middleware),
});

const { useGetTicketsQuery, useCreateTicketMutation } = api;

function Tickets() {
  const { data, isLoading, isError, error, refetch } = useGetTicketsQuery();
  const [createTicket, { isLoading: isSaving }] = useCreateTicketMutation();
  const {
    register, handleSubmit, reset, setError, clearErrors,
    formState: { errors },
  } = useForm<TicketInput>({ defaultValues: { title: "" } });
```

```
  async function onSubmit(values: TicketInput) {
    clearErrors("root.server");
    try {
      await createTicket({ title: values.title.trim() }).unwrap();
      reset();
    } catch (failure: unknown) {
      setError("root.server", { type: "server", message: describeError(failure)
});
    }
  }

  return (
    <section>
      <h1>Tickets</h1>
      {isLoading ? <p>Loading...</p> : isError ? (
        <div role="alert">
          <p>{describeError(error)}</p>
          <button type="button" onClick={() => void refetch()}>Retry</button>
        </div>
      ) : data ? (
        data.length === 0
          ? <p>No tickets yet.</p>
          : <ul>{data.map(ticket => <li key={ticket.id}>{ticket.title}</li>)}</ul>
      ) : <p role="alert">No response data received.</p>}

      <form onSubmit={handleSubmit(onSubmit)} noValidate>
        <label htmlFor="title">Title</label>
        <input
          id="title"
          aria-invalid={Boolean(errors.title)}
          {...register("title", {
            validate: value => value.trim().length > 0 || "Title is required.",
          })}
        />
        {errors.title && <p role="alert">{errors.title.message}</p>}
        {errors.root?.server && <p role="alert">{errors.root.server.message}</p>}
        <button type="submit" disabled={isSaving}>
          {isSaving ? "Saving..." : "Create"}
        </button>
      </form>
    </section>
  );
}

function describeError(error: unknown): string {
  if (typeof error === "object" && error !== null && "status" in error) {
    return `Request failed (${String(error.status)}).`;
  }
  return "Request failed. Please try again.";
}

export default function App() {
```

```
  return <Provider store={store}><Tickets /></Provider>;
}
```

The mutation invalidates the ticket tag so active queries can refetch; `.unwrap()` makes the submission failure observable to the form. This example does not validate response JSON at runtime or configure authentication. In production, add contract validation, authorization, a suitable API base URL and CORS policy, and a safe strategy for ambiguous create failures before allowing retries. An error after a timeout does not establish whether the ticket was created.

Applying this to actual experience

Review a feature you really worked on that involved React, TypeScript, RTK, React Hook Form, or an API. Identify which tools were actually used, how data flowed from input to request to UI, which errors and validation rules mattered, and what you personally changed. If a tool was not part of that feature, discuss the example as a proposed design rather than claiming prior implementation.

Likely interview questions

1. What does Redux Toolkit add over writing Redux setup by hand?
2. How can `createSlice` use mutation-like syntax without directly mutating the prior state?
3. When would you keep state local rather than put it in Redux?
4. How does RTK Query cache and invalidate server data?
5. What does `.unwrap()` provide when handling a mutation?
6. How does React Hook Form handle registered inputs and validation?
7. Why must the server repeat validation and authorization?
8. What happens if a create request times out after the server may have committed it?

Common pitfalls / misconceptions

- Putting every form field and transient UI value in global state.
- Assuming a TypeScript response type validates network data.
- Displaying an empty list as if it were a successful response when the request actually failed.
- Retrying an ambiguous non-idempotent mutation without server-side deduplication.
- Claiming experience with implementation details that were only part of an illustrative example.

Cheat-sheet recap

- RTK simplifies store and reducer setup; RTK Query manages server-state caching and request status.
- React Hook Form manages form values, validation, and submission feedback.
- Handle loading, empty, success, and failure states explicitly; validate on the server.
- Design cache invalidation, authentication, runtime validation, and mutation retries intentionally.